

SZEGEDI TUDOMÁNYEGYETEM

Természettudományi és Informatikai Kar

Informatikai Intézet

SZAKDOLGOZAT

ProdCty - audiofókuszú közösségi platform producereknek és zenészeknek

Készítette: Tompa Márton

Gazdaságinformatikus Bsc.

Témavezető: Dr. Bilicki Vilmos

egyetemi docens

Szeged

2026

Tartalmi összefoglaló

A szakdolgozat egy audiofókuszú közösségi webalkalmazás, a ProdCty megtervezését és megvalósítását mutatja be. A projekt célja egy olyan MVP létrehozása volt, amely producereknek, zenészeknek és zenei alkotóknak ad felületet sample-ek és demók feltöltésére, meghallgatására, keresésére, értékelésére, valamint közösségi visszajelzésre. A rendszer külön kezeli a sample és demo típusú hanganyagokat: a sample-ek letölthető könyvtári elemekként, a demók pedig visszajelzésre és együttműködésre szánt munkaverzióként jelennek meg.

A megvalósítás **React**, **TypeScript** és **Vite** alapú frontendből, **Node.js** és **Express** alapú backendből, valamint **MongoDB** adatbázisból áll. A felhasználói azonosítás JWT tokenekkel történik, a médiafájlok lokálisan vagy production környezetben **Cloudinary** segítségével tárolhatók. A rendszer része a felhasználói profilkezelés, a sample library, a demo board, az aura-alapú vizuális hangulatjelzés, a rating és feedback popup, a kommentválaszok, a collab request és inbox, valamint az adminisztrátori moderáció.

A dolgozat külön kitér a tesztelésre, az online üzemeltetésre és a mesterséges intelligencia használatára. Az MI a fejlesztési folyamatban fejlesztői segédeszközként jelent meg, emellett a kész rendszerben is szerepet kap sample risk screening formájában. Az alkalmazás OpenAI API-val vagy szabályalapú fallback logikával jelzi az adminisztrátor számára a licenc vagy spam szempontból gyanús sample feltöltéseket. Az eredmény egy működő, online is publikálható MVP, amely nem teljes értékű piaci szolgáltatás, de bemutatja egy audioalapú közösségi platform legfontosabb technikai és terméklogikai alapjait.

Kulcsszavak: audio platform, React, TypeScript, Node.js, MongoDB, mesterséges intelligencia

Tartalom

Tartalmi összefoglaló	1
1. Bevezetés.....	4
1.1. Motiváció	5
1.2. A dolgozat célja és határai	6
2. Területi és technológiai áttekintés.....	6
2.1. Hasonló rendszerek	6
3. Követelmények és termékterv	9
3.1. Nem funkcionális követelmények	10
3.2. Felhasználói utak.....	10
4. Rendszerterv	11
4.1. Architektúra.....	11
4.2. Adatmodell	12
4.3. API-csoportok	13
4.4. Üzleti szabályok	14
5. Megvalósítás.....	14
5.1. Autentikáció és profilkezelés	14
5.2. Sample library	15
5.3. Demo board és Aura.....	15
5.4. Feedback és kommentválaszok	16
5.5. Collab request és inbox	16
5.6. Adminisztrátori moderáció.....	17
6. Mesterséges intelligencia a kész rendszerben	17
7. Tesztelés és validáció	18
7.1. Fejlesztés közbeni ellenőrzés	18

7.2. Automatikus tesztek	18
7.3. Manuális E2E ellenőrzés	19
7.4. Lefedettség és korlátok.....	20
8. Online üzemeltetés	21
9. Mesterséges intelligencia használata a fejlesztés során	22
10. Értékelés, korlátok és továbbfejlesztési lehetőségek.....	24
11. Összefoglalás.....	25
Irodalomjegyzék.....	27
Nyilatkozat	28

1. Bevezetés

A digitális zenei alkotás az elmúlt években sokkal szélesebb körben elérhetővé vált. Egy kezdő vagy haladó producer ma már otthoni környezetben is képes olyan hanganyagokat létrehozni, amelyek korábban stúdióhoz, drága hardverekhez vagy komolyabb infrastruktúrához kötődtek. A digitális audio munkaállomások, pluginek, sample packok és online megosztófelületek miatt a zenei alkotás technikai belépési küszöbe alacsonyabb lett, ugyanakkor a közösségi visszajelzés és a minőségi anyagok megtalálása továbbra is nehéz feladat.

A legtöbb online platform nem kifejezetten produceri munkafolyamatokra épül. Egy általános közösségi oldalon a zenei tartalom gyakran videós, képi vagy algoritmikus figyelemért versengő formában jelenik meg. A producer számára viszont sokszor nem ez a lényeg, hanem az, hogy gyorsan meg tudjon hallgatni egy ötletet, visszajelzést kapjon egy demóra, vagy találjon egy használható dob, vokál, gitár vagy ambient hangmintát. Ebből a gyakorlati igényből indult ki a ProdCty projekt.

A ProdCty egy audiofókuszú közösségi platform producereknek és zenészeknek. A rendszer célja nem az volt, hogy egy kész, üzletileg skálázott Splice- vagy SoundCloud-szerű szolgáltatást helyettesítsen, hanem hogy szakdolgozati keretek között bemutassa egy ilyen irányú termék MVP szintű megvalósítását. Az MVP kifejezés itt azt jelenti, hogy a rendszer a legfontosabb felhasználói folyamatokat valósítja meg, miközben bizonyos nagyobb volumenű irányokat, például a fejlett ajánlórendszert vagy a teljes audiofeldolgozó pipeline-t tudatosan későbbi fejlesztésként kezeli.

1.1. Motiváció

A projekt motivációja részben személyes érdeklődésből, részben egy gyakorlati problémából indult ki. Zenei alkotás közben gyakran előfordul, hogy egy félkész ötlethez jó lenne gyors, őszinte és konkrét visszajelzést kapni. Saját tapasztalatból is éreztem ezt a hiányt, mert nincs sok olyan zenész vagy producer ismerősöm, akinek rendszeresen meg tudnám mutatni a készülő projektjeimet. Emiatt sokszor nehéz eldönteni, hogy egy demóban működik-e a hangulat, elég erős-e a groove, jó irányba megy-e a mix, vagy maga az ötlet érdemes-e további kidolgozásra.

Egy demó esetében nem feltétlenül az a cél, hogy azonnal publikálható szám legyen belőle, hanem az, hogy az alkotó visszajelzést kapjon a munka aktuális állapotáról. Az általános közösségi platformok erre kevésbé alkalmasak, mert ott a zenei tartalom gyakran más típusú figyelemért versenyez, és nem kifejezetten szakmai vagy alkotói visszajelzésre épül. A ProdCty egyik alapötlete ezért az volt, hogy legyen egy olyan felület, ahol a demók nem végleges megjelenésként, hanem visszajelzésre szánt munkaverzióként jelennek meg.

A másik motiváció a sample-ek kezelése volt. Korábban volt lehetőségem kipróbálni a Splice-ot is, mert egy ismerősöm fiókját kölcsön tudtam használni. A felület és a keresési logika nagyon hasznosnak tűnt, főleg azért, mert gyorsan lehetett hangulat, műfaj vagy konkrét hangtípus alapján sample-eket találni. Ugyanakkor azt is láttam, hogy ez egy fizetős szolgáltatás, ami kezdő vagy hobbiszinten alkotó producereknek nem mindig ideális megoldás. Innen jött az ötlet, hogy érdemes lenne egy olyan, egyszerűbb sample library irányt is megvizsgálni, ahol a feltöltött hanganyagok műfaj, típus, energia, BPM, hangnem vagy dobtípus alapján kereshetők. Egy producer számára például más jelentése van annak, hogy valami loop vagy one-shot, kick vagy snare, rock vagy lofi hangulatú. Ezek a kategóriák a ProdCty adatmodelljében és felületén is megjelennek.

1.2. A dolgozat célja és határai

A dolgozat célja a ProdCty webalkalmazás tervezésének és megvalósításának bemutatása. A hangsúly azon van, hogyan lehet egy audiofókuszú közösségi ötletből működő szoftveres rendszert kialakítani: milyen entitásokra, API-kra, frontend nézetekre, jogosultságokra, tesztelési lépésekre és üzemeltetési döntésekre van szükség.

A projekt határait tudatosan kellett meghúzni. A kezdeti ötletben több nagy komplexitású funkció is felmerült, például TikTok-szerű audio feed, professzionális CDN-alapú médiastreaming, mélyebb audioanalitika vagy teljes értékű ajánlórendszer. Ezek önmagukban is komoly fejlesztési feladatok, ezért a végleges MVP a következő fő területekre koncentrál: regisztráció és login, sample library, demo board, audiolejátszás, feedback, collab request, profilkezelés, admin moderáció, online deploy és AI-alapú sample risk screening.

Ez a döntés szakmai szempontból fontos volt, mert egy szakdolgozati projekt értékelésekor nem az a legfontosabb, hogy minél több funkció szerepeljen a tervben, hanem az, hogy a vállalt funkciók bizonyíthatóan működjenek, dokumentálva legyenek és illeszkedjenek egymáshoz. A dolgozat ezért nem egy ideális jövőbeli terméket ír le, hanem a ténylegesen elkészült rendszerre fókuszál.

2. Területi és technológiai áttekintés

2.1. Hasonló rendszerek

A ProdCty problématerét több ismert zenei platform érinti. A sample könyvtárak elsősorban hangminták keresésére, meghallgatására és letöltésére szolgálnak, míg a zenei közösségi platformok inkább publikálásra, követésre és visszajelzésre épülnek. A két világ között van átfedés, de a felhasználói szándék nem azonos. Egy sample könyvtárban a felhasználó általában gyorsan szeretne megtalálni egy újrahasználató hangot, egy demo felületen viszont azt szeretné, hogy a saját munkaverziójára kapjon véleményt.

A ProdCty ebből a kettősségből indul ki. A sample-eknél a keresés, szűrés, letöltés és jogtisztaság hangsúlyos. A demóknál az aura-megjelenítés, a lejátszás, az 1-5 pontos feedback, a kommentválasz és a collab request kap fontosabb szerepet. Ezzel a rendszer nem egyetlen nagy

feedbe keveri az összes hanganyagot, hanem a tartalom célja szerint választja szét az interakciókat.

2.2. Modern webalkalmazási modell

A ProdCty kliens-szerver architektúrában készült. A frontend a böngészőben futó React alkalmazás, a backend pedig Express API-kat biztosít. Ez a felosztás jól illeszkedik a modern webalkalmazásokhoz, mert a felhasználói felület és az üzleti logika külön rétegben fejleszthető. A frontend nem közvetlenül az adatbázissal kommunikál, hanem HTTP-kéréseket küld a backendnek, amely ellenőrzi a bemenetet, a jogosultságokat és a tartalomtípushoz tartozó szabályokat.

A frontend React és TypeScript alapú. A React komponensalapú szemlélete jól használható olyan felületeknél, ahol ismétlődő elemek jelennek meg, például track sorok, lejátszóblokkok, profilkártyák, modal ablakok vagy admin sorok [1]. A TypeScript statikus típusellenőrzést ad, ami különösen hasznos akkor, amikor a frontend többféle API-választ, felhasználói állapotot és track típust kezel [2]. A Vite gyors fejlesztői szerveret és build folyamatot biztosít, ami a React átállás és a folyamatos finomítás során sokat gyorsított [3].

A backend Node.js és Express környezetben készült. A Node.js alkalmas HTTP API-k és I/O-intenzív alkalmazások futtatására [4], az Express pedig egyszerű, route-okra és middleware-ekre épülő webes keretrendszert ad [5]. A ProdCty esetében ez a szervezés áttekinthetővé teszi az auth, track, user, collab és admin funkciók elkülönítését.

2.3. Adattárolás és hitelesítés

Az adattárolás MongoDB és Mongoose segítségével történik. A MongoDB dokumentumalapú adatbázis, amely rugalmasan kezeli a változó szerkezetű, egymáshoz kapcsolódó adatokat [6]. A ProdCty adatmodellje több olyan entitást tartalmaz, amelyeknél a dokumentumalapú modell praktikus: felhasználó, track, komment, rating, report, collab request és vote. A Mongoose sémák, validációk és modellek segítségével strukturáltabb módon használható a MongoDB Node.js alkalmazásból [7].

A hitelesítés JSON Web Token alapú. Bejelentkezés után a backend aláírt tokent ad vissza, amelyet a frontend a védett kéréseknél használ. A JWT formátum célja, hogy kompakt módon hordozzon ellenőrizhető állításokat a felhasználóról [8]. A ProdCty esetében a token tartalmazza

a felhasználó azonosítóját, nevét és szerepkörét. A backend minden védett műveletnél ellenőrzi, hogy a token érvényes-e, az admin funkcióknál pedig külön szerepkör-ellenőrzés is történik.

2.4. Online környezet és médiafájlok

A projekt célja kezdettől nem egy kizárólag lokálisan futtatható alkalmazás elkészítése volt, hanem egy online is elérhető webalkalmazás létrehozása. A lokális futtatás főként fejlesztési és tesztelési szerepet töltött be: így gyorsabban lehetett kipróbálni az új funkciókat, hibákat keresni és javításokat ellenőrizni anélkül, hogy minden kisebb módosítást azonnal commitolni és deployolni kellett volna. Ez különösen a frontend finomhangolásánál, a feltöltési folyamatoknál és az admin funkciók tesztelésénél volt hasznos.

A végleges irány ezzel szemben online környezetre épül. A frontend Vercelre, a backend Renderre, az adatbázis MongoDB Atlasra, a médiafájlok pedig Cloudinary tárhelyre helyezhetők. Ez azért fontos, mert egy szakdolgozati bemutatónál és egy termékszerű alkalmazásnál is előny, ha a rendszer nem csak a fejlesztő saját gépén működik. A Render web service modellje alkalmas Node.js API futtatására [11], a Vercel pedig React/Vite frontendek publikálására [12]. A MongoDB Atlas felhős adatbázist ad [13], a Cloudinary pedig médiafájlok tárolását és kiszolgálását támogatja [14].

A médiafájlok kezelése különösen fontos audioalkalmazásnál. Fejlesztői környezetben a feltöltések lokális uploads mappába kerülhetnek, mert ez gyors és egyszerű megoldás teszteléshez. Éles környezetben viszont ez nem elég stabil, mert a platformszolgáltatók fájlrendszere nem feltétlenül tartós tároló. Emiatt a projektben szerepel **Cloudinary** integráció, amely production beállítás esetén a feltöltött audio- és avatarfájlokat külső médiaszolgáltatóhoz tölti fel, az adatbázisban pedig csak az URL és a kapcsolódó metaadat marad.

3. Követelmények és termékterv

A követelmények kialakításakor a legfontosabb szempont az volt, hogy a rendszer ne csak technikai demó legyen, hanem használható termékírányt mutasson. Emiatt a funkciókat nem önmagukban, hanem felhasználói folyamatok szerint rendeztem. A felhasználó regisztrál, megadja érdeklődési köreit, sample-t vagy demót tölt fel, hanganyagot hallgat, visszajelzést ad, együttműködést kezdeményez vagy reportot küld. Az adminisztrátor ezzel párhuzamosan ellenőrzi a gyanús tartalmakat, reportokat és problémás felhasználókat.

1. táblázat: A ProdCty fő funkcionális követelményei

Azonosító	Követelmény	Leírás	Állapot
F1	Regisztráció és login	Felhasználó létrehozása, belépés, JWT alapú munkamenet, erősebb jelszóminimum.	Megvalósult
F2	Érdeklődési körök	Regisztrációnál és profilnál zenei műfajok megadása, amely a demo ajánlásnál is használható.	Megvalósult
F3	Sample library	Sample feltöltés, szűrés, keresési javaslatok, lejátszás, letöltés, jogtisztasági nyilatkozat.	Megvalósult
F4	Demo board	Demo feltöltés, aura waveform, My Style és Top Voted nézet, lejátszás.	Megvalósult
F5	Feedback	1-5 pontos demo értékelés, szöveges visszajelzés, válaszok és felhasználói profilhivatkozások.	Megvalósult
F6	Collab request	Demóhoz kapcsolódó együttműködési kérés, inbox, accept/decline, opcionális Instagram kontakt.	Megvalósult
F7	Report és admin	Upload és komment jelentése, admin review, felhasználói warning/ban, track és komment törlés.	Megvalósult
F8	AI sample screening	Sample metaadatok kockázati előszűrése OpenAI API-val vagy szabályalapú fallbackkel.	Megvalósult
F9	Online publikálás	Vercel frontend, Render backend, Atlas DB, Cloudinary	Megvalósult

3.1. Nem funkcionális követelmények

A nem funkcionális követelmények közül a használhatóság, a karbantarthatóság és az alapvető biztonság kapott nagyobb szerepet. Az alkalmazásnak áttekinthetőnek kell lennie, mert a célcsoport nem adminisztratív feladatot akar végezni, hanem gyorsan hanganyagot keresni, feltölteni vagy értékelni. A frontend ezért több helyen **popup** alapú feltöltést és visszajelzést használ, hogy a fő lista nézetek ne legyenek túlszűfoltak.

Karbantarthatósági szempontból fontos volt a frontend és backend szétválasztása. A backend route-ok külön fájlokban vannak, a frontend pedig oldalakra és komponensekre bontva tartalmazza a fő logikát. A típusos frontend adatszerkezetek segítenek abban, hogy az API-válaszok ne keveredjenek össze, például egy sample sor ne próbáljon demo rating logikát használni.

Biztonsági szempontból a projekt nem teljes körű production security auditot valósít meg, de több alapvető megoldást tartalmaz. A jelszavak hash-elve tárolódnak, az auth végpontok egyszerű rate limitinget használnak, az admin funkciókat role check védi, a publikus keresés nem ad vissza email címet, és a collab requestben megadott Instagram elérés csak elfogadott kérés után látható. A biztonsági szemlélethez az OWASP Top 10 általános irányai is kapcsolhatók [16].

3.2. Felhasználói utak

A felhasználói utak tervezésénél a fő cél az volt, hogy a rendszerben ne különálló funkciók legyenek, hanem végigjárható folyamatok. Egy új felhasználó először regisztrál, megadja érdeklődési köreit, majd a Library oldalon sample-eket böngészhet. Ha feltölt sample-t, először audiofájlt választ, megadja a címet, műfajokat, opcionális BPM-et és hangnemet, majd a tényleges feltöltés előtt el kell fogadnia a **jogtisztasági nyilatkozatot**. Ez a lépés azért került be, mert a sample tartalmaknál a felhasználási jog kérdése különösen érzékeny.

A demo folyamat más logikát követ. A felhasználó munkaverziót tölt fel, megadja a műfajokat, a rendszer pedig a metaadatok alapján aura színvilágot generál. A demo nem letöltésre szolgál, hanem meghallgatásra, értékelésre és együttműködésre. A felhasználó a 'Demos' oldalon elindíthatja a tracket, majd feedback popupban értékelheti, válaszolhat mások visszajelzésére, vagy collab requestet küldhet.

Az adminisztrátori út ettől eltérő, mert nem tartalomfogyasztásra, hanem moderációra épül. Az admin látja az overview statisztikákat, az open reportokat, a reviewed ügyeket, a trackeket, a kommenteket, a felhasználókat és külön az **AI review listát**. Ha egy sample gyanús, az admin meghallgathatja mini lejátszóval, törölheti, clear-elheti az AI flaget, vagy törlés mellett warningot/ban-t adhat a feltöltőnek.

4. Rendszerterv

4.1. Architektúra

A rendszer három fő rétegből áll: frontend, backend és adatbázis/médiatárolás. A frontend React alkalmazás a felhasználói interakciókat kezeli. A backend Express API-kat ad a frontendnek, és felel a jogosultságokért, validációkért, adatbázisműveletekért és médiafájl-kezelésért. Az adatbázis MongoDB, a médiafájlok fejlesztés közben lokálisan, production környezetben pedig Cloudinaryban tárolódnak.

2. táblázat: A rendszer fő rétegei

Réteg	Technológia	Felelősség
Frontend	React, TypeScript, Vite	Nézetek, komponensek, API-hívások, lejátszó, modalok, route transition.
Backend	Node.js, Express	REST API, auth, business rules, fájlfeltöltés, report, collab, admin műveletek.
Adatbázis	MongoDB Atlas / lokális MongoDB, Mongoose	Felhasználók, trackek, kommentek, ratingek, reportok, collab requestek és vote-ok tárolása.
Média	Lokális uploads / Cloudinary	Audio- és avatarfájlok tárolása, production környezetben külső media storage.
Deploy	Render, Vercel	Backend és frontend online publikálása GitHub alapú deploymenttel.

A frontend és backend közötti kommunikáció REST jellegű API-hívásokon keresztül történik. A frontend a VITE_API_URL környezeti változóból tudja, hogy production környezetben melyik backend URL-t kell hívnia. A backend CORS beállítással szabályozza, hogy mely frontend domainek férhetnek hozzá. Ez különösen akkor lényeges, amikor a frontend Vercelen, a backend pedig Renderen fut.

4.2. Adatmodell

Az adatmodell központi eleme a Track entitás, mert minden audio tartalom ehhez kapcsolódik. A Track kind mezője dönti el, hogy sample-ről vagy demóról van szó. Ez a mező nem csak megjelenítési célú, hanem üzleti szabályok alapja: a sample letölthető és jogtisztasági nyilatkozatot igényel, a demo viszont ratinget, feedbacket és collab requestet fogad.

3. táblázat: A fő adatmodell entitásai

Entitás	Szerep	Fontosabb mezők
User	Felhasználói profil és jogosultság	username, email, passwordHash, role, moderationStatus, warningCount, interests, bio, avatarUrl
Track	Feltöltött audio tartalom	kind, genre, tags, bpm, musicalKey, aura, licenseConfirmed, aiRiskLevel, audioUrl, playCount, vote count
Comment	Szöveges visszajelzés és válasz	trackId, parentId, parentRatingId, userId, author, category, text, isDeleted
Rating	Demo feedback pontszámmal	trackId, userId, author, score, text, updatedAt
Report	Moderációs jelentés	targetType, trackId, commentId, reporterId, reason, status, resolutionNote
CollabRequest	Együttműködési kérés	trackId, requesterId, trackOwnerId, message, skills, contactPreference, status
TrackVote	Fel- és leszavazás	trackId, userId, value, updatedAt

A kommenteknél külön figyelmet kapott a válaszolhatóság. A Comment modell parentId mezője kommentre adott választ, a parentRatingId pedig ratinghez kapcsolódó választ jelöl. Ez lehetővé teszi, hogy a feedback popupban a pontszámos értékelések és a szöveges válaszok egy közös, hierarchikus timeline-ban jelenjenek meg.

4.3. API-csoportok

A backend API-k route csoportokra vannak bontva. Az authRoutes kezeli a regisztrációt és bejelentkezést, a userRoutes a profilokat és avatarokat, a trackRoutes a hanganyagokat, a ratingeket, kommenteket, vote-okat és reportokat, a collabRoutes az együttműködési kéréseket, az adminRoutes pedig a moderációs felületet. Ez a felosztás segíti, hogy egy-egy üzleti területhez tartozó logika ne keveredjen össze.

4. táblázat: Fő API-csoportok

Csoport	Példák	Funkció
Auth	/auth/register, /auth/login	Felhasználó létrehozása, token kiadása, banned állapot kezelése.
Tracks	/tracks, /demos, /feed/for-you	Hanganyagok listázása, feltöltése, streamingje, letöltése és szűrése.
Feedback	/tracks/:id/ratings, /tracks/:id/comments	Demo értékelés, komment, válasz és saját komment törlése.
Reports	/tracks/:id/reports, /comments/:id/reports	Upload és komment jelentése admin review-ra.
Collab	/tracks/:id/collab-requests, /collab-requests	Collab request küldése, inbox listázás, elfogadás/elutasítás.
Admin	/admin/overview, /admin/reports, /admin/users, /admin/tracks	Moderáció, AI review, user warning/ban, törlések.

4.4. Üzleti szabályok

A ProdCty egyik lényeges része, hogy nem minden tartalomnál ugyanazok a műveletek engedélyezettek. A demók nem tölthetők le, mert céljuk a visszajelzés és a kollaboráció. A sample-ek letölthetők, de feltöltéskor jogtisztasági nyilatkozatot kell elfogadni. Rating kizárólag demókra adható, míg upvote és downvote mindkét tartalomtípusnál működik.

A moderációs szabályoknál a rendszer figyeli a felhasználók warning count értékét. Egy figyelmeztetés után a felhasználó belépéskor account warning bannert lát. Két figyelmeztetés után a rendszerben állapotba teszi, és az uploadjai törlésre kerülhetnek. Az admin önmagát és más admin fiókokat nem moderálhat, mert ez hibás vagy veszélyes adminisztrátori állapothoz vezetne.

A jogtisztasági popup szintén üzleti szabály. A sample feltöltése előtt a felhasználónak el kell fogadnia, hogy csak olyan hanganyagot tölt fel, amelynek megosztásához joga van. Ez önmagában nem bizonyíték, de a rendszer működésében fontos felelősségvállalási pont. Az AI risk screening ezt egészíti ki azzal, hogy metaadatok alapján gyanús mintákat jelöl admin review-ra.

5. Megvalósítás

5.1. Autentikáció és profilkezelés

A regisztráció során a felhasználó felhasználónevet, email címet, jelszót és érdeklődési köröket adhat meg. A backend ellenőrzi a felhasználónév hosszát, az email formátumát, a jelszó minimum hosszát, valamint azt, hogy az email vagy a felhasználónév nem foglalt-e. A jelszó bcryptjs segítségével hash-elve kerül az adatbázisba.

Bejelentkezéskor a backend ellenőrzi a megadott email-jelszó párost. Siker esetén JWT token állít elő, amely a frontend oldali munkamenet alapja. Tiltott felhasználó esetén a login nem engedélyezett, és a felhasználó egyértelmű hibaüzenetet kap. Figyelmeztetett felhasználó esetén a belépés megtörténhet, de a felületen account warning jelenik meg.

A profilkezelés két nézetből áll: saját profilból és publikus profilból. A saját profilban módosítható a bio, az érdeklődési kör és az avatar. A publikus profil más felhasználók sample- és

demo feltöltéseit mutatja, de email címet nem ad vissza. Ez egyszerű adatvédelmi döntés, de egy közösségi alkalmazásban fontos, mert a kereső és a profiloldal publikusabb felület.

5.2. Sample library

A Library nézet célja, hogy a felhasználók gyorsan találjanak sample jellegű hanganyagokat. A keresőmező a Splice-szerű mintából indult ki: gépeléskor javaslatokat ad, üres keresőnél pedig népszerű kategóriákat jelenít meg. A drum keresésnél a rendszer nem csak a drum szóra keres, hanem ide veszi a kick, snare, hihat, clap, rim, perc, tom, cymbal és 808 címkéket is. Ez azért praktikus, mert egy producer gyakran nem általános dobot, hanem konkrét dobhangot keres.

A sample feltöltése popup ablakban történik. A felhasználó kiválasztja az audiofájlt, megadja a címet, legfeljebb három műfajt, opcionális BPM-et, hangnemet, dobtípust és sample típust. A rendszer nem próbál BPM-et tippelni, ha a felhasználó nem adja meg, mert a gyakorlatban a fájlnevből vagy címből tippelt BPM sokszor pontatlan lehet. Ez egy tudatos visszalépés a túlzott automatizálástól: jobb üres vagy ismeretlen értéket mutatni, mint hibás metainformációt.

A sample-eknél piros waveform preview segíti a gyors böngészést. A play gomb lejátszáskor pause állapotba vált, és a bottom playerben is megjelenik az aktuális hanganyag. A sample letölthető, de csak akkor kerülhet fel, ha a felhasználó elfogadta a jogtisztasági popupt. A report gombbal a felhasználók spam, copyright, explicit, misleading vagy other ok alapján jelenthetnek problémás feltöltést.

5.3. Demo board és Aura

A Demos oldal a work-in-progress hanganyagokra épül. A demo feltöltése szintén popup ablakban történik, de itt a hangsúly nem a letöltésen, hanem a visszajelzésen van. A felhasználó kiválasztja a fájlt, megadja a címet, műfajokat, opcionális BPM-et és hangnemet. A demók legfeljebb öt percesek lehetnek, hogy a felület ne hosszú, teljes albumjellegű anyagokra, hanem konkrét, visszajelzésre alkalmas részletekre fókuszáljon.

Az Aura funkció a demo hangulatát vizuálisan jelöli. A rendszer a megadott műfaj, energia, cím, fájlnev és egyéb metaadatok alapján generál színpalettát. A cél nem valós audioanalízis vagy zenei érzelemfelismerés, hanem egy könnyen érthető, vizuális azonosító, amely megkülönbözteti a demókat. A műfajokhoz vibráns színek kapcsolódnak, de az azonos műfajú demók sem teljesen egyformák, mert a hash-alapú eltérés módosítja a palettát.

A demo feltöltésnél az aura nem azonnal jelenik meg véglegesnek. A felhasználó a submit után rövid elemzési állapotot lát, majd körülbelül másfél másodperc után színesedik be a preview. Ez felhasználói élmény szempontjából azért fontos, mert a feltöltő visszajelzést kap arról, hogy a rendszer feldolgozza az adatokat, és nem csak statikusan kirajzol egy színt.

5.4. Feedback és kommentválaszok

A demók értékelése széles feedback popupban történik. A korábbi elképzelésben külön komment és review blokk is szerepelt, de ez redundánsnak bizonyult. A végleges megoldásban a pontszámas feedback és a szöveges válaszok egy közös timeline-ban jelennek meg. A felhasználó 1-5 közötti pontszámot adhat, és mellé rövid megjegyzést írhat. Más felhasználók erre vagy egy sima kommentre is válaszolhatnak.

A válaszok hierarchikusan, beljebb húzva jelennek meg, és látható, hogy ki kinek válaszolt. A kommentelő profilképe és neve hivatkozásként működik a publikus profilra. A saját komment törölhető, de a törlés nem bontja szét a beszélgetést: a helyén jelzés marad, hogy a komment törölve lett. Ez a megoldás megőrzi a reply szálak érthetőségét.

A feedback funkció tudatosan csak demókhoz kapcsolódik. Sample-eknél nincs rating popup, mert ott a felhasználói cél más: a sample-t keresni, meghallgatni és letölteni szeretnék. Ez a döntés csökkenti a felület zajosságát, és világosabbá teszi a két tartalomtípus közötti különbséget.

5.5. Collab request és inbox

A collab request funkció arra szolgál, hogy egy felhasználó ne csak passzív visszajelzést adjon, hanem együttműködést is kezdeményezzen. A demó sorában lévő Collab gombra kattintva popup nyílik, ahol a felhasználó megadhatja, miben tudna segíteni, például gitár, vokál, mix, mastering, beat vagy songwriting területen. Emellett üzenetet és kontaktpreferenciát adhat meg.

Az Instagram elérés külön adatvédelmi szabályt kapott. Ha a kérés Instagram kontaktot tartalmaz, azt a demo tulajdonosa csak elfogadás után láthatja. Ez azért hasznos, mert a kapcsolatfelvétel nem válik automatikusan nyilvánossá, és a felhasználó csak akkor adja át a közvetlenebb kontaktját, ha a másik fél ténylegesen érdeklődik.

Az inbox a fejlécből érhető el. Itt a felhasználó látja a bejövő és kimenő collab requesteket, valamint dönthet az elfogadásról vagy elutasításról. A funkció az MVP-ben egyszerű, de jól mutatja, hogy a rendszer nem csak tartalomlistázó, hanem közösségi interakciókat is kezel.

5.6. Adminisztrátori moderáció

Az admin felület célja, hogy a közösségi működéshez szükséges alapvető moderációs eszközöket biztosítsa. Az admin overview statisztikákat mutat a felhasználókról, sample-ekről, demókról, kommentekről, ratingekről, play countokról, upvote/downvote számokról és open reportokról. A moderációs tabok külön kezelik az open reportokat, a lezárt review-kat, az AI review listát, a trackeket, a kommenteket és a felhasználókat.

Az admin a reportoknál eldöntheti, hogy a jelentés invalid, reviewed vagy actioned státuszba kerüljön. A trackek és kommentek törölhetők, a felhasználók figyelmeztethetők, bannolhatók vagy clear-elhetők. Két warning után automatikus ban léphet életbe. Bannolt felhasználó esetén a rendszer a feltöltéseit is törli, hogy a problémás tartalom ne maradjon bent. Fontos védelmi szabály, hogy admin nem moderálhatja saját magát, és admin fiókok nem jelennek meg moderálható felhasználóként.

Az AI review külön tabot kapott, mert a gyanús sample-ek nem azonosak a felhasználói reportokkal. A reportot egy felhasználó indítja, az AI review viszont a feltöltés metaadatai alapján keletkezik. Az admin a suspicious sample-t mini lejátszóval meghallgathatja, clear-elheti az AI flaget, vagy review action popupban törölheti a feltöltést, opcionális warninggal vagy bannal.

6. Mesterséges intelligencia a kész rendszerben

A ProdCty kész rendszerében az MI nem általános chatbotként vagy marketingcélú extraként jelenik meg, hanem konkrét moderációs feladatot támogat. A sample feltöltésekor a backend a megadott metaadatokat elküldheti OpenAI API-nak, amely strukturált JSON választ ad arról, hogy a feltöltés metaadatai alapján van-e licenc, spam vagy tisztázatlan forrás kockázat. Az OpenAI API használata a hivatalos dokumentáció alapján történik [15].

Az AI sample risk screening nem jogi döntés. Nem azt mondja ki, hogy egy sample biztosan jogsértő vagy biztosan jogtiszt, hanem azt, hogy a metaadatok alapján indokolt-e adminisztrátori review. Gyanús jelzés lehet például a YouTube, ripped, official, remix, stem, acapella, ismert előadó neve, vagy harmadik féltől származó sample pack említése. Ezek a szavak nem bizonyítanak jogsértést, de jelzik, hogy az adminnak érdemes ránéznie a feltöltésre.

A rendszer úgy készült, hogy az AI API hiánya ne tegye működésképtelenné a feltöltést. Ha nincs OpenAI API kulcs, vagy az API-hívás hibázik, szabályalapú fallback fut le. Ez a fallback egyszerű reguláris minták alapján keresi a gyanús kifejezéseket. A megoldás így demonstrálja az AI-alapú működést, de közben robusztusabb is, mert nem függ teljesen egy külső szolgáltatás pillanatnyi elérhetőségétől.

A feltöltés előtt a jogtisztasági popup és az AI review egymást kiegészíti. A popup a felhasználói felelősségvállalást rögzíti, az AI screening pedig admin oldali előszűrést ad. Egy szakdolgozati MVP-ben ez reális kompromisszum: nem próbál automatikus szerzői jogi döntést hozni, de bemutatja, hogyan lehet AI-t értelmesen beépíteni egy platform moderációs folyamatába.

7. Tesztelés és validáció

7.1. Fejlesztés közbeni ellenőrzés

A fejlesztés elején a tesztelés főként lokálisan történt. A backend a helyi MongoDB adatbázissal futott, a frontend Vite dev szerverrel indult. Ez a környezet gyors iterációt tett lehetővé: egy-egy módosítás után rögtön ellenőrizhető volt a regisztráció, a login, a feltöltés, a lejátszás vagy az admin funkció. A lokális tesztelés különösen fontos volt a frontend kialakításánál, mert több elrendezést és interakciós mintát kellett kipróbálni.

Később a validáció kiterjedt az online környezetre is. A backend Renderen, a frontend Vercelen, az adatbázis MongoDB Atlason, a médiafájlok Cloudinaryban futtathatók. Ez új típusú hibákat is felszínre hozott: CORS beállítás, környezeti változók, production API URL, felhős média URL-ek, Atlas IP hozzáférés és deploy logok ellenőrzése. Ezek a lépések a projekt életútja szempontjából lényegesek, mert a rendszer már nem csak a fejlesztői gépen működik.

7.2. Automatikus tesztek

A backend teszteléséhez Jest és Supertest került felhasználásra. A Jest a JavaScript tesztfutató keretrendszer szerepét tölti be [9], a Supertest pedig HTTP API-k tesztelését segíti [10]. A tesztek mockolt modellekkel futnak, így nem kell valós adatbázis minden route viselkedésének ellenőrzéséhez.

Az utolsó lokális tesztfuttatásban három tesztsomag futott le sikeresen: app routes, trackService és audio aura generation. Összesen **21 teszt** futott, mindegyik sikeres eredménnyel. A tesztek

ellenőrzik például a publikus user search email nélküli választ, a play count explicit növelését, a jelszóvalidációt, az admin jogosultságkezelést, a második warning utáni bant, az admin önmoderáció tiltását, a demo listázást, valamint az aura generálás alaplogikáját.

5. táblázat: Tesztelési rétegek

Réteg	Eszköz	Ellenőrzött terület
Backend route tesztek	Jest, Supertest	Auth, admin, track, feed, jogosultságok, hibás bemenetek.
Üzleti logika teszt	Jest	Track validáció, addTrack, filterByGenre alaplogika.
Aura teszt	Jest	Műfaj és energia alapján generált színek, BPM-tippelés mellőzése.
Frontend build	TypeScript, Vite	Típushibák, buildelhetőség, production bundle.
Frontend lint	ESLint	Kódszintű stílus- és hibajelzések.
Manuális E2E	Böngésző, lokális és online környezet	Regisztráció, upload, feedback, collab, admin, deploy működés.

7.3. Manuális E2E ellenőrzés

A frontendnél a manuális E2E smoke tesztelés különösen fontos maradt, mert több funkció vizuális és interakciós jellegű. Ellenőrizni kellett, hogy a guest felhasználó upload gombra kattintva az auth oldalra kerül-e, a sample feltöltés csak jogtisztasági elfogadás után történik-e, a demo aura a megfelelő állapotváltással jelenik-e meg, a play/pause ikon helyesen vált-e, illetve a feedback popupban működik-e a válaszolás.

Az admin felületnél külön regressziós pont volt, hogy az admin ne tudja saját magát warningolni vagy törölni. Ez egy fejlesztés közben előjött hiba volt, amely jól mutatja, hogy a jogosultságkezelésnél nem elég csak a tipikus user-admin különbséget nézni. Az admin saját fiókja speciális eset, ezért backend és frontend oldalon is védeni kellett. A javításhoz automatikus teszt is készült.

Az online validáció során a legfontosabb ellenőrzések a következők voltak: a Render backend /health végpontja elérhető, a Vercel frontend a production API URL-t hívja, a regisztráció és login működik, sample és demo feltöltés után a médiafájl lejátszható, avatar megjelenik, a Cloudinary URL-ek működnek, és az admin felület látja az AI review-ra kerülő sample-eket.

7.4. Lefedettségi és korlátok

A backend coverage riport alapján az automatikus tesztek nem fedik le a teljes kódbázist. Ez az MVP állapotában nem meglepő, mert a projektben sok olyan rész van, amely fájlfeltöltéshez, média storage-hoz vagy böngészős interakcióhoz kötődik. A meglévő tesztek értéke abban van, hogy a főbb hibás jogosultsági és validációs eseteket lefedik, de a későbbi fejlesztéshez érdemes lenne növelni az integrációs és E2E tesztek arányát.

A következő lépés egy Playwright alapú frontend E2E tesztcsomag lehetne. Ez automatikusan ellenőrizhetné a regisztrációt, sample feltöltést, search suggestion működést, demo aura megjelenést, feedback popupot, collab inboxot és admin review folyamatot. Ehhez külön tesztadatbázis és seedelt felhasználók szükségesek, hogy a tesztek ismételhetők legyenek, és ne módosítsák az éles bemutató adatokat.

8. Online üzemeltetés

A konzulensi elvárás szempontjából fontos, hogy a projekt ne csak lokálisan legyen futtatható, hanem online is publikálható termékként jelenjen meg. A ProdCty esetében a végleges irány az lett, hogy a frontend Vercelen, a backend Renderen, az adatbázis MongoDB Atlason, a médiafájlok pedig Cloudinaryban legyenek kezelve. Ez a felosztás a szakdolgozati projekt méretéhez képest reális, mert mindegyik szolgáltatás jól illeszkedik az adott réteghez.

6. táblázat: Online környezet

Komponens	Szolgáltatás	Beállítás
Frontend	Vercel	Root directory: frontend-react, build: npm run build, output: dist, VITE_API_URL a Render backend URL-re.
Backend	Render Web Service	Root directory: backend, build: npm ci, start: npm start, production környezeti változók.
Adatbázis	MongoDB Atlas	Felhős MongoDB connection string, adatbázisnév: prodcty, IP access list.
Média	Cloudinary	Audio és avatar feltöltések production környezetben külső media storage-ba kerülnek.
Forráskód	GitHub	main branch push után automatikus deployment vagy manuális redeploy.

Az online deploy során több gyakorlati probléma is előjött. A Render root directory beállításának pontosnak kellett lennie, különben a szolgáltatás nem találta a backend mappát. A MongoDB Atlasnál engedélyezni kellett a megfelelő IP-hozzáférést. A Vercelnél a VITE_API_URL változónak a Render backend URL-jére kellett mutatnia. Emellett a CORS_ORIGIN értéket is be kellett állítani a backend oldalon, hogy a Vercelről érkező kérések engedélyezettek legyenek.

A médiafájloknál a Cloudinary integráció azért jelentett előrelépést, mert production környezetben a lokális fájlrendszer nem megfelelő hosszabb távú tárolásra. A backend a feltöltött fájlokat Cloudinaryba küldi, majd a MongoDB-ben a média URL-je és metaadatai tárolódnak. Ez

a megoldás nem professzionális, nagy terhelésre optimalizált média streaming infrastruktúra, de szakdolgozati MVP szinten jól demonstrálja a felhős tárolás és a backend integráció működését.

9. Mesterséges intelligencia használata a fejlesztés során

A ProdCty fejlesztése során a mesterséges intelligenciát két fő szerepben használtam. Egyrészt fejlesztői segédeszközként, amely segített a kódolásban, hibakeresésben, refaktorálásban, dokumentációban és döntési alternatívák átgondolásában. Másrészt a kész alkalmazásba is bekerült egy konkrét AI-alapú funkció, a sample risk screening, amely a sample feltöltések metaadatait vizsgálja, és gyanús esetben admin review-ra jelöli őket.

A fejlesztési folyamatban leggyakrabban ChatGPT-t és Codex jellegű kódasszisztenst használtam. A ChatGPT inkább magyarázatban, hibák értelmezésében, dokumentációs szerkezetben és alternatívák keresésében segített. A Codex erősebb szerepet kapott a konkrét kódbázis olvasásában, fájlok módosításában, route-ok, komponensek, tesztek és dokumentációs fájlok összehangolásában. A két eszközt nem ugyanarra használtam: az egyik inkább gondolkodási és kommunikációs partner volt, a másik inkább implementációs segéd.

Az MI-t kódgenerálásra főleg kisebb, jól körülhatárolt feladatoknál használtam. Ilyen volt például egy validációs ág megírása, egy API-hívás bekötése, egy modal állapotkezelése, vagy egy admin route jogosultsági ellenőrzése. A nagyobb funkciókat több lépésre kellett bontani. Például a collab request nem egyetlen promptból lett kész: külön kellett kezelni a backend modellt, a route-ot, az inbox listázást, az accept/decline logikát, az Instagram láthatósági szabályt és a frontend popupt.

A frontendnél az MI használata több odafigyelést igényelt. Gyakran előfordult, hogy a javasolt megoldás technikailag működőképes lett volna, de nem egyezett az elképzelt felhasználói élménnyel. Például több iteráció kellett ahhoz, hogy a Demos oldal ne legyen redundáns a korábbi Community nézetrel, a feedback ne külön komment és review blokkban jelenjen meg, hanem egy közös timeline-ban, és az action gombok ne zsúfolódjanak össze. Ilyenkor a promptokat többször pontosítani kellett: nem elég azt írni, hogy legyen szebb vagy felhasználóbarátabb, hanem meg kellett fogalmazni, hogy pontosan melyik elem hol legyen, milyen szélességgel, milyen interakcióval és milyen állapotokkal.

A backend logikánál az MI általában könnyebben használható kiindulási pontot adott, mert ott jobban körülhatárolható szabályokról volt szó. Például egy route-nál meg lehet adni, hogy milyen bemenetet vár, milyen hibakódot adjon vissza, milyen Mongoose modellt módosítson, és milyen jogosultság kell hozzá. Ennek ellenére itt sem lehetett automatikusan elfogadni a javaslatokat. Többször előfordult, hogy az MI más mezőnévvel vagy egyszerűbb adatstruktúrával számolt, mint amit a projekt ténylegesen használt. Ilyenkor kézzel kellett hozzáigazítani a User, Track, Comment, Rating, Report vagy CollabRequest modellhez.

A validálásnál az MI kimenetét mindig hipotézisként kezeltem. Egy javasolt kód akkor került be, ha futott, illeszkedett a meglévő struktúrához, és manuálisan vagy automatikus tesztel ellenőrizhető volt. Backend módosítások után futtattam a Jest teszteket és a syntax checket, frontend módosítások után a TypeScript buildet és a lintet. A vizuális módosításokat böngészőben néztem meg, mert egy frontend elrendezési hibát nem feltétlenül kap el a fordító.

Konkrét példa volt az admin önmoderáció hibája. A rendszer egy korábbi állapotban lehetővé tette, hogy az admin saját magát warningolja. Ez nem okozott azonnali technikai összeomlást, de terméklogikailag hibás és veszélyes működés. A javítás során nem csak a frontend listából kellett kiszűrni az admin saját fiókját, hanem backend oldalon is tiltani kellett a saját admin és más admin fiókok moderálását. Ehhez külön regressziós teszt készült. Ez jól mutatja, hogy az MI-segítség mellett is a fejlesztő felelőssége észrevenni a nem triviális jogosultsági eseteket.

Az MI-t dokumentációhoz is használtam, de ott különösen figyelni kellett a pontosságra. Egy dokumentációs szöveg könnyen jól hangzik akkor is, ha nem teljesen igaz. Emiatt a README, UX dokumentáció, deploy guide és szakdolgozati fejezetek esetén folyamatosan össze kellett vetni a leírtakat a repó tényleges fájljaival és a működő alkalmazással. Például nem lehetett azt állítani, hogy kész egy fejlett ajánlórendszer, ha a jelenlegi For You logika egyszerű pontozásra épül. Ugyanígy a média storage-nál külön kellett választani a lokális fejlesztési és production Cloudinary működést.

Nem használtam MI-t arra, hogy a projekt alapötletét, célcsoportját vagy fő termékdöntéseit helyettem meghozza. A sample és demo tartalmak szétválasztása, a feedback-fókuszú demo board, az aura vizuális irány és a produceri célcsoport saját döntés volt. Az MI ezek megfogalmazásában, finomításában és implementációs részleteiben segített, de a végső scope és prioritás fejlesztői döntés maradt.

A folyamatból azt tanultam, hogy az MI akkor hasznos igazán, ha pontos kérdést kap és van mihez igazodnia. Minél konkrétan adtam meg a fájlokat, az elvárt viselkedést, a meglévő modelleket és a nem kívánt mellékhatásokat, annál jobb választ adott. Amikor túl általános volt a prompt, a válasz is általános lett. Ez különösen frontendnél volt látványos: a látványról, elrendezésről és használhatóságról sokkal részletesebben kellett kommunikálni, mint egy backend validációról.

Összességében az MI gyorsította a fejlesztést, de nem váltotta ki a tervezést, tesztelést és ellenőrzést. A következő projektben még korábban vezetnék pontosabb acceptance criteriákat és prompt/verification naplót, mert így utólag is jobban követhető lenne, hogy melyik MI-javaslat milyen döntéshez vezetett. A ProdCty esetében a legfontosabb tanulság az volt, hogy az MI nem kész megoldásokat ad, hanem egy munkaverziót, amit mérnöki szemmel kell ellenőrizni és a saját termékhez igazítani.

10. Értékelés, korlátok és továbbfejlesztési lehetőségek

A ProdCty eredménye egy működő, online is publikálható MVP. A rendszer megvalósítja a regisztrációt, login folyamatot, sample és demo feltöltést, audiolejátszást, keresést, szűrést, feedbacket, kommentválaszt, collab requestet, profilkezelést, reportolást, admin moderációt és AI sample risk screeninget. Ezek együtt már nem pusztán prototípust, hanem egy végigjárható termékszeletet alkotnak.

A projekt egyik erőssége, hogy a sample és demo tartalmak elkülönítése nem csak felületi döntés, hanem végigmegy a modellen, API-n és üzleti szabályokon. Ez teszi a rendszert koherenssé. A sample-eknél jogtisztaság, letöltés és kereshetőség fontos, a demóknál visszajelzés, aura, rating és collab. A kettő összekeverése rontaná a felhasználói élményt, ezért a megvalósításban ez végig külön maradt.

A másik erősség az admin és AI review réteg. Egy közösségi alkalmazásnál a feltöltött tartalmak kezelése nem állhat meg annál, hogy a felhasználók bármit feltölthetnek. A report, AI suspicious flag, warning, ban, komment törlés és track törlés együtt már mutat egy alapvető moderációs folyamatot. Ez nem teljes jogi vagy biztonsági megoldás, de szakdolgozati MVP szinten fontos termékminőségi elem.

A korlátok között első helyen az ajánlórendszer egyszerűsége áll. A My Style nézet jelenleg a felhasználó érdeklődési köreihez és a track metaadataihoz kapcsolódó pontozással működik, nem tanuló modell alapján. A Top Voted nézet objektívebb rendezést ad, de nem veszi figyelembe a mélyebb hallgatási szokásokat. Később érdemes lenne részletesebb ajánlási logikát, user activity alapú scoringot vagy gépi tanulási irányt kipróbálni.

A médiafeldolgozás szintén továbbfejleszthető. Jelenleg az aura metaadatokra épül, nem valós audio feature extractionre. Egy későbbi verzióban lehetne BPM-detektálás, hangnemfelismerés, waveform valódi amplitúdó alapján, automatikus csendvágás vagy sample slicing. Ezek azonban már külön audiofeldolgozási problémák, és nagyobb tesztelést igényelnének.

A tesztelésben a következő nagy lépés a Playwright alapú E2E automatizálás lenne. A backend route tesztek hasznosak, de a frontend legfontosabb értéke az interakciókban van: upload popup, feedback modal, collab inbox, admin review action, theme váltás, keresési javaslatok. Ezeket hosszabb távon nem elég manuálisan ellenőrizni, mert könnyű regressziókat bevinni egy új UI-módosítással.

Üzemeltetési szempontból a jelenlegi **Render/Vercel/Atlas/Cloudinary** megoldás szakdolgozati bemutatóra megfelelő, de nagyobb felhasználószámnál további lépések kellenének: részletesebb logging, rate limiting, storage quota, vírusellenőrzés, moderációs audit log, backup stratégia, részletesebb adatvédelmi tájékoztató és erősebb admin hozzáférés-kezelés.

11. Összefoglalás

A szakdolgozatban bemutatott ProdCty projekt célja egy audiofókuszú közösségi webalkalmazás létrehozása volt producerek és zenészek számára. A rendszer két fő tartalomtípust kezel: sample-eket és demókat. A sample-ek letölthető, kereshető hangminták, a demók pedig visszajelzésre és együttműködésre feltöltött munkaverziók. Ez a különbségtétel a projekt legfontosabb terméklogikai döntése.

A megvalósítás React, TypeScript, Vite, Node.js, Express, MongoDB és Mongoose technológiákra épül. A rendszer JWT alapú autentikációt, role-alapú admin funkciókat, Cloudinary-alapú production médiatárolást, Render és Vercel alapú online publikálást, valamint OpenAI API-val vagy fallback szabályrendszerrel működő sample risk screeninget tartalmaz. A

frontend komponensekre bontott, a backend route-csoportokba szervezett, az adatmodell pedig a közösségi audio platform fő entitásait írja le.

A fejlesztés során fontos tanulság volt, hogy egy termékminőségű szakdolgozati projekt nem csak funkeiőlistából áll. A működő kód mellett szükség van követelményekre, dokumentációra, tesztekre, validációra, deployra, jogosultsági szabályokra és ismert korlátok vállalására. A ProdCty jelenlegi állapotában nem teljes értékű piaci szolgáltatás, de egy koherens, bemutatatható MVP, amely lefedi az ötlet legfontosabb felhasználói és technikai folyamatait.

A projekt továbbfejleszthető fejlettebb ajánlórendszerrel, valódi audioanalízissel, automatizált frontend E2E tesztekkel, részletesebb admin audit loggal és robusztusabb médiafeldolgozással. A jelen dolgozat célja azonban az volt, hogy a megvalósított rendszer jelenlegi, működő állapotát mutassa be. Ezt a célt a ProdCty teljesíti: regisztrálható, használható, online publikálható, hanganyagokat kezel, közösségi visszajelzést támogat, és rendelkezik moderációs és AI-alapú előszűrési réteggel is.

Irodalomjegyzék

- [1] React documentation: Learn React. <https://react.dev/learn> Utolsó megtekintés: 2026. május 2.
- [2] TypeScript documentation. <https://www.typescriptlang.org/docs/> Utolsó megtekintés: 2026. május 2.
- [3] Vite documentation: Getting Started. <https://vite.dev/guide/> Utolsó megtekintés: 2026. május 2.
- [4] Node.js documentation. <https://nodejs.org/en/docs> Utolsó megtekintés: 2026. május 2.
- [5] Express documentation. <https://expressjs.com/> Utolsó megtekintés: 2026. május 2.
- [6] MongoDB Manual. <https://www.mongodb.com/docs/manual/> Utolsó megtekintés: 2026. május 2.
- [7] Mongoose documentation. <https://mongoosejs.com/docs/> Utolsó megtekintés: 2026. május 2.
- [8] Jones, M., Bradley, J., Sakimura, N. 2015. JSON Web Token (JWT). RFC 7519. <https://www.rfc-editor.org/rfc/rfc7519> Utolsó megtekintés: 2026. május 2.
- [9] Jest documentation: Getting Started. <https://jestjs.io/docs/getting-started> Utolsó megtekintés: 2026. május 2.
- [10] Supertest package documentation. <https://www.npmjs.com/package/supertest> Utolsó megtekintés: 2026. május 2.
- [11] Render documentation: Web Services. <https://render.com/docs/web-services> Utolsó megtekintés: 2026. május 2.
- [12] Vercel documentation: Deployments. <https://vercel.com/docs/deployments> Utolsó megtekintés: 2026. május 2.
- [13] MongoDB Atlas documentation. <https://www.mongodb.com/docs/atlas/> Utolsó megtekintés: 2026. május 2.
- [14] Cloudinary documentation. <https://cloudinary.com/documentation> Utolsó megtekintés: 2026. május 2.
- [15] OpenAI API documentation. <https://platform.openai.com/docs/> Utolsó megtekintés: 2026. május 2.
- [16] OWASP Foundation: OWASP Top 10. <https://owasp.org/www-project-top-ten/> Utolsó megtekintés: 2026. május 2.
- [17] GitHub Actions documentation. <https://docs.github.com/en/actions> Utolsó megtekintés: 2026. május 2.
- [18] Tompa Márton: ProdCty szakdolgozati projekt forráskódja. GitHub repository.
<https://github.com/Mtompero/ProdCty> Utolsó megtekintés: 2026. május 3.
- [19] Tompa Márton: ProdCty online alkalmazás.
<https://prod-cty.vercel.app> Utolsó megtekintés: 2026. május 3.

Nyilatkozat

Alulírott Tompa Márton gazdaságinformatikus BSc szakos hallgató kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem Informatikai Intézetében készítettem, gazdaságinformatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat, szakirodalmat, dokumentációkat és eszközöket használtam fel. Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Szeged, 2026. május 3.

Aláírás